

AgroScan — Guia Técnico

Roteiro de estudo para a apresentação com o professor · foco no código e nas decisões de engenharia (não no pitch de startup)

1. Arquitetura geral

Resposta rápida se ele perguntar "como é organizado?": Expo + React Native + TypeScript, com navegação em 3 camadas, estado global via Context API e persistência local via AsyncStorage. Sem backend — tudo roda no dispositivo.

App.tsx	-> carrega fontes, monta os providers, renderiza o navigator
src/	
navigation/	-> RootNavigator -> MainTabs -> MoreNavigator (3 camadas)
context/AppContext.tsx	-> estado global (Context API + hooks) + AsyncStorage
screens/	-> 1 componente por tela, sem lógica de negócio pesada
components/	-> peças reutilizáveis (Button, Card, ScreenHeader, StatusBadge)
data/	-> dados mockados (pragas, setores, ordens, histórico seed)
theme/	-> design tokens (cores, tipografia) extraídos do protótipo
types/	-> interfaces TypeScript compartilhadas

Fluxo de dados: tela → hook useApp() → funções que alteram estado → AsyncStorage persiste → Context notifica re-render nas telas que usam aquele dado.

2. Decisões técnicas e a justificativa de cada uma

Por que Expo (não React Native puro)?

Managed workflow — não precisa configurar Xcode/Android Studio/Gradle na mão. expo-camera e expo-image-picker já vêm com config plugin pronto (ver app.json). Corta muito tempo de setup para um projeto de validação rápida. Trade-off: menos controle sobre código nativo customizado.

Por que TypeScript?

Todas as entidades do domínio (praga, setor, ordem de serviço, usuário) estão tipadas em src/types/index.ts. Isso pega erro de propriedade errada em tempo de compilação — por exemplo, Severity é um union type ('critical' | 'moderate' | 'low'), então é impossível passar uma severidade inválida sem o TS reclamar.

Por que Context API e não Redux/Zustand?

O estado é pequeno e sem lógica assíncrona complexa (sem múltiplas fontes de dados, sem cache complicado). Um AppProvider com useState + useCallback resolve sem a boilerplate de reducers/actions. Se o app crescesse para sync real com backend, aí sim valeria migrar para Zustand ou React Query.

Por que React Navigation com navegadores aninhados?

RootNavigator decide Login vs. App logado (stack). Dentro do app logado, MainTabs é a bottom tab bar (Home/Scanner/Maps/Reports/Mais). "Mais" é ele mesmo outra stack (MoreNavigator) porque tem sub-telas que precisam de botão voltar. Padrão comum de apps reais: tabs no nível raiz,

stacks dentro de cada tab quando necessário.

Por que AsyncStorage?

Simples, key-value, suficiente para persistir sessão de login e histórico de detecções entre aberturas do app. Não precisa de banco relacional (SQLite) porque não há consultas complexas — é só salvar/recuperar um array.

Por que react-native-svg no mapa?

Para desenhar um gradiente radial de verdade (heatmap suave) em vez de uma View com `backgroundColor` translúcido, que fica com borda dura. `<RadialGradient>` do SVG dá um degradê contínuo — ver `MapScreen.tsx`.

3. A "IA" é simulada — seja transparente sobre isso

Se ele perguntar "cadê o modelo de IA": não existe modelo real embarcado. Em `ScannerScreen.tsx`, a função `runAnalysis` sorteia uma praga de um catálogo fixo (`data/pests.ts`) depois de ~1.8s de "análise" simulada.

Isso é proposital nesta etapa: o objetivo agora é validar a **experiência do app** (fluxo, UX, navegação), não o modelo de visão computacional em si — que seria a próxima etapa técnica (provavelmente TensorFlow Lite ou ML Kit rodando on-device, treinado com imagens rotuladas de pragas).

4. Perguntas técnicas prováveis + resposta curta

Pergunta	Resposta
Como funciona a permissão de câmera?	<code>useCameraPermissions()</code> do expo-camera; se não concedida, mostra tela pedindo permissão com fallback para galeria (expo-image-picker).
Como os dados persistem entre sessões?	AsyncStorage, chaves <code>agrosan:history</code> , <code>agrosan:user</code> , <code>agrosan:orders</code> — ver <code>AppContext.tsx</code> .
Por que o status da Ordem de Serviço é clicável nos 3 estados?	Decisão de UX: status editável nas duas direções (não só avançar), com confirmação (<code>Alert.alert</code>) antes de marcar como concluído, para evitar erro do usuário.
O app funciona sem internet?	Estruturalmente sim (não depende de rede para funcionar), mas a detecção real de online/offline ainda não está implementada; é um próximo passo (<code>NetInfo</code>).
Como está tipada a navegação?	<code>RootStackParamList</code> , <code>MainTabParamList</code> , <code>MoreStackParamList</code> em <code>navigation/types.ts</code> — o TypeScript valida os parâmetros de cada rota (ex: <code>ReportDetail</code> exige <code>{ id: string }</code>).
Existem testes automatizados?	Ainda não — é um MVP de validação de UX. Próximo passo natural seria testes de componente com Jest + Testing Library.
Por que separar <code>theme/colors.ts</code> e <code>theme/typography.ts</code> ?	Design tokens extraídos 1:1 do protótipo (Stitch/Figma) — cor e fonte centralizadas, qualquer ajuste de marca muda em um lugar só.

5. Se ele pedir para editar algo ao vivo

Mostre o Fast Refresh: abra um arquivo simples como `DashboardScreen.tsx`, mude um texto, salve (`Ctrl+S`), e o app atualiza no celular sem reiniciar. É um bom "wow" técnico rápido e mostra que você entende o ciclo de desenvolvimento.